

第24讲核心概念速查卡

一个神经元的 Backpropagation

核心概念：速查卡

本讲把“节点级 backward”组装成一个真正的神经元反向传播。目标不是背公式，而是理解每个 Weight、Bias、Input 为什么会得到对应的梯度，以及这些梯度的 Shape 为什么与原变量一致。



建议学习方式：先跟着一个 3 输入神经元手算 -> 再看向量公式 -> 最后运行 Gradient Check 与 PyTorch Autograd 对照。

1. Forward

$$z = x \cdot w + b$$

$$a = \text{ReLU}(z)$$

$$L = (a - y)^2$$

2. Backward 主链

$$g_a = 2(a - y)$$

$$g_z = g_a * \text{ReLU}'(z)$$

3. 参数 / 输入梯度

$$dL/dw = g_z * x$$

$$dL/db = g_z$$

$$dL/dx = g_z * w$$

4. 本讲数字答案

$x=[2,-1,3]$, $w=[0.5,-2,1]$, $b=0.2$, $y=5$

$z=6.2$, $a=6.2$, $L=1.44$

$dL/dz=2.4$

$dL/dw=[4.8,-2.4,7.2]$

$dL/db=2.4$

$dL/dx=[1.2,-4.8,2.4]$

5. Shape

量	Shape
x / w / dL/dx / dL/dw	(D,)
b / z / a / L / dL/db / dL/dz	scalar

6. 必须记住的直觉

Weight gradient: 上游梯度 $\times$ 对应输入。

Input gradient: 上游梯度 $\times$ 对应 Weight。

Bias gradient: 直接等于上游梯度 dL/dz 。

ReLU 关闭时, 这条路径的 dL/dz 可以变成0。

梯度 Shape 与被求导变量 Shape 对应。

Gradient 不是参数更新量; 还要乘 learning rate / 经过 optimizer。

7. PyTorch 映射

```
loss.backward()  
print(w.grad)  
print(b.grad)  
print(x.grad)
```